

HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT ET2100

ONE LOVE. ONE FUTURE.



Chương VI : *Các giải thuật tìm kiếm*

Giảng viên: TS. Đỗ Thị Ngọc Diệp
Khoa Kỹ thuật Truyền thông – Trường Điện Điện Tử

ONE LOVE. ONE FUTURE.

1. Bài toán tìm kiếm
2. Các giải thuật tìm kiếm phần tử bằng so sánh
 - Giải thuật tìm kiếm tuần tự (Sequential searching algorithm)
 - Giải thuật tìm kiếm nhị phân (Binary searching algorithm)
3. Tìm kiếm chuỗi con
4. Các giải thuật tìm kiếm phần tử trực tiếp (Direct searching)
 - Đặt vấn đề
 - Hàm băm (Hash functions)
 - Các biện pháp khắc phục đụng độ

1. Đặt vấn đề

- **Bài toán tìm kiếm:** 2 bài toán tìm kiếm cơ bản:
 - Tìm một phần tử trong một dãy phần tử cho trước :
 - Tìm kiếm bằng cách so sánh
 - Tìm kiếm trực tiếp từ giá trị khoá
 - Tìm sự xuất hiện của một chuỗi con trong một chuỗi cho trước
 - Giải thuật tìm kiếm đơn giản (còn gọi là tìm kiếm thô)
 - Giải thuật phức tạp: Knuth-Morris-Pratt, Boyer-Moore, etc.

Giải thuật tìm kiếm phần tử

- Bài toán cơ sở:
 - Cho một dãy N số: $A = (a_0, a_1, \dots, a_{N-1})$ và giá trị cần tìm K (khoá tìm kiếm).
 - Yêu cầu tìm vị trí của phần tử có giá trị bằng K.
- Tìm kiếm bằng so sánh
 - So sánh K với lần lượt các phần tử trong dãy cần tìm cho đến khi ra kết quả (hoặc tìm thấy hoặc không tìm thấy)
 - 2 loại giải thuật tìm kiếm theo cách này:
 - Tìm kiếm tuần tự (Sequential Search)
 - Tìm kiếm nhị phân (Binary Search)
- Tìm kiếm trực tiếp
 - Sử dụng hàm băm: giúp tính trực tiếp địa chỉ của phần tử cần tìm từ giá trị của phần tử

2. Các giải thuật tìm kiếm bằng so sánh

(1) Tìm kiếm tuần tự

- Ý tưởng giải thuật:

- Để tìm phần tử bằng K trong dãy N số: $A = (a_0, a_1, \dots, a_{N-1})$, tiến hành so sánh K với **lần lượt các phần tử** trong dãy, cho đến khi:
 - Hoặc tìm thấy phần tử $a_i = K$, thì trả về vị trí i cần tìm
 - Hoặc đã so sánh với toàn bộ các phần tử của dãy nhưng vẫn không thấy, thì trả về kết quả không tìm thấy.

```
typedef int DaySo[N];
```

```
int TKTuanTu(DaySo a, int K) {  
    int i=0;  
    while (i<N && a[i] != K) i++;  
    if (i<N) return i;    //Tìm thấy  
    return -1;            //Không tìm thấy  
}
```

2. Các giải thuật tìm kiếm bằng so sánh

(1) Tìm kiếm tuần tự

- Đánh giá giải thuật
 - Trường hợp tốt nhất: $O(1)$
 - Trường hợp xấu nhất: $O(n)$
 - Trường hợp trung bình: $O(n)$

2. Các giải thuật tìm kiếm bằng so sánh

(2) Tìm kiếm nhị phân

- Ý tưởng giải thuật đệ quy:

- Để tìm phần tử bằng K trong dãy N số $A = (a_0, a_1, \dots, a_{N-1})$, với điều kiện dãy A đã được sắp xếp (giả sử theo chiều tăng dần):
 - So sánh K với phần tử a_m ở giữa dãy ($m = \lfloor N/2 \rfloor$).
 - Nếu $K = a_m \rightarrow$ vị trí tìm thấy m
 - Nếu $K < a_m \rightarrow$ Tìm K trong dãy $(a_0, a_1, \dots, a_{m-1})$
 - Nếu $K > a_m \rightarrow$ Tìm K trong dãy $(a_{m+1}, a_{m+2}, \dots, a_{N-1})$
 - Điểm dừng: khi tìm thấy hoặc khi dãy không còn phần tử nào thì trả về kết quả không tìm thấy.

2. Các giải thuật tìm kiếm bằng so sánh

(2) Tìm kiếm nhị phân

- Ý tưởng giải thuật đệ quy:

```
int BSearch (DaySo a, int K, int b, int e) {  
    if (b>e) return -1;  
    int m = (b+e)/2;  
    if (K==a[m]) return m;  
    else  
        if (K<a[m])  
            return BSearch(a, K, b, m-1);  
        else  
            return BSearch(a, K, m+1,e);  
}  
int BinarySearch(DaySo a, int K){  
    return BSearch(a,K,0,a.length-1);  
}
```

2. Các giải thuật tìm kiếm bằng so sánh

(2) Tìm kiếm nhị phân

- Đánh giá giải thuật
 - Trường hợp tốt nhất: $O(1)$
 - Trường hợp xấu nhất: $O(\log_2 n)$
 - Trường hợp trung bình: $O(\log_2 n)$

3. Tìm kiếm chuỗi con

- Bài toán:
 - Cho trước một văn bản V gồm n kí tự ($v_0, v_1, \dots, v_{n-1}$) và một chuỗi con P (gọi là mẫu) gồm m kí tự ($p_0, p_1, \dots, p_{m-1}$).
 - Yêu cầu tìm vị trí xuất hiện đầu tiên [tất cả] của P trong V
- Ví dụ
 - V : “the **rain** in sp**ain** stays **mainly** on the **plain**”
 - P : “ain”
- Ứng dụng:
 - Trong tìm kiếm thông tin
 - Soạn thảo văn bản
 - Xử lý dữ liệu sinh học (ADN)
- Các giải thuật tìm kiếm chuỗi con:
 - Giải thuật tìm kiếm thô (Brute-Force)
 - Giải thuật Knuth-Morris-Pratt (KMP)

3. Tìm kiếm chuỗi con

- Giải thuật tìm kiếm thô (Brute-Force)
 - Ý tưởng:
 - So khớp mẫu với văn bản theo từng ký tự trong mẫu từ trái sang phải
 - Khi có một vị trí không khớp được xác định, đẩy toàn bộ mẫu sang phải 1 vị trí và tiếp tục so khớp với văn bản theo cách thức trên

P:	p_0	p_1	...	p_i	...	p_{m-1}		int BFSearch (char v[], int N, char p[], int M)
				⌘				{
V:	v_0	...	v_j	v_{j+1}	...	v_{j+i}	...	v_{n-1}
								if (M>N) return -1;
								int i=0, j=0;
								while (j<N){
								while (i<M && j<N && p[i]==v[i+j])
								i++;
								if (i==M) return j;
								i=0;
								j++;
								}
								return -1;

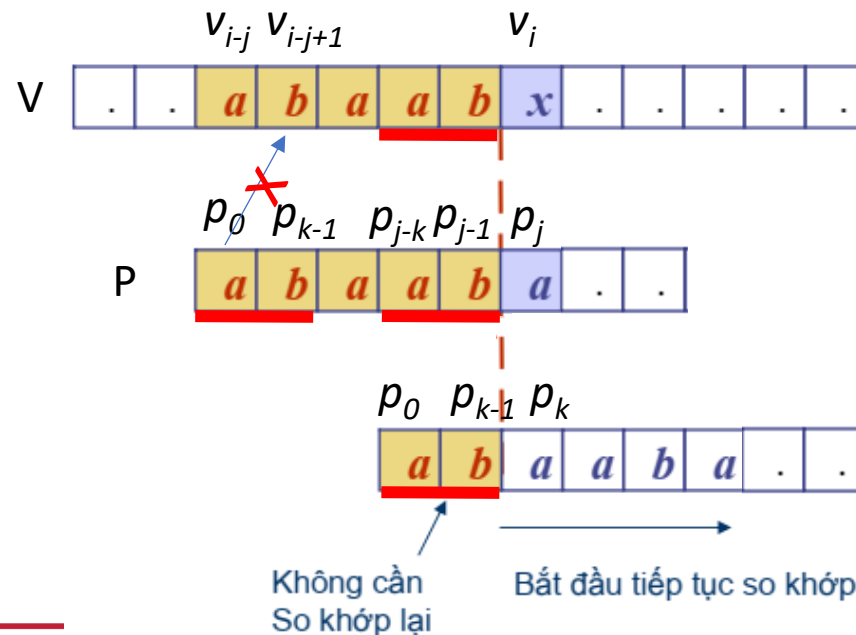
- Đánh giá giải thuật: Trường hợp xấu nhất: số thao tác so sánh: $M*N \rightarrow O(N^2)$

3. Tìm kiếm chuỗi con

- Giải thuật Knuth-Morris-Pratt

- Ý tưởng:

- Cải tiến từ giải thuật tìm kiếm thô
- Khi so sánh đến cặp ký tự (v_i, p_j) đầu tiên mà $v_i \neq p_j$ (j cặp trước đó đã khớp)
- Thay vì quay lại so sánh p_0 của mẫu với ký tự v_{i-j+1}
→ tiếp tục việc so sánh từ một ký tự p_k trong mẫu với ký tự v_i hiện đang không khớp trong văn bản



3. Tìm kiếm chuỗi con

- Giải thuật Knuth-Morris-Pratt

- Vị trí k cần tìm trong mẫu P thỏa mãn các điều kiện:

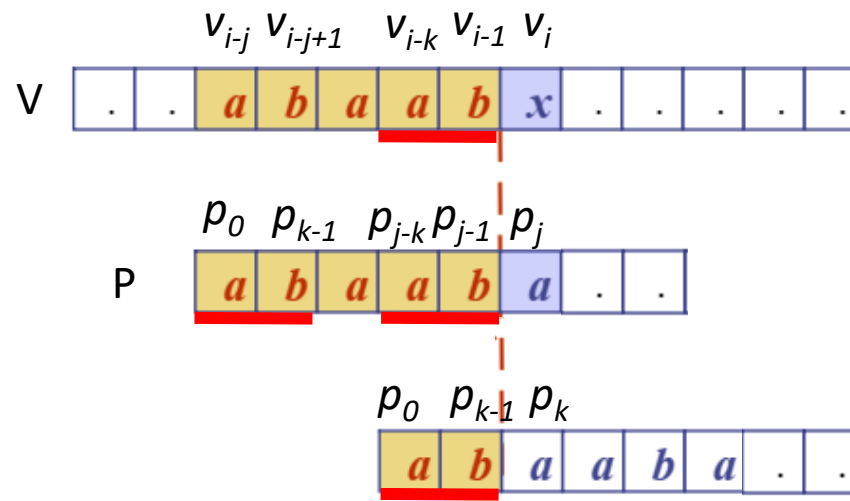
- Lớn nhất có thể: $k < j$, $p_k \neq p_j$;

- $p_0 p_1 \dots p_{k-1} = v_{i-k} \dots v_{i-1}$;

- Tính chất của k :

- k phụ thuộc vào vị trí j và nội dung mẫu, không phụ thuộc vào văn bản

- $[p_0 p_1 \dots p_{k-1}] = [p_{j-k} p_{j-k+1} \dots p_{j-1}]$



3. Tìm kiếm chuỗi con

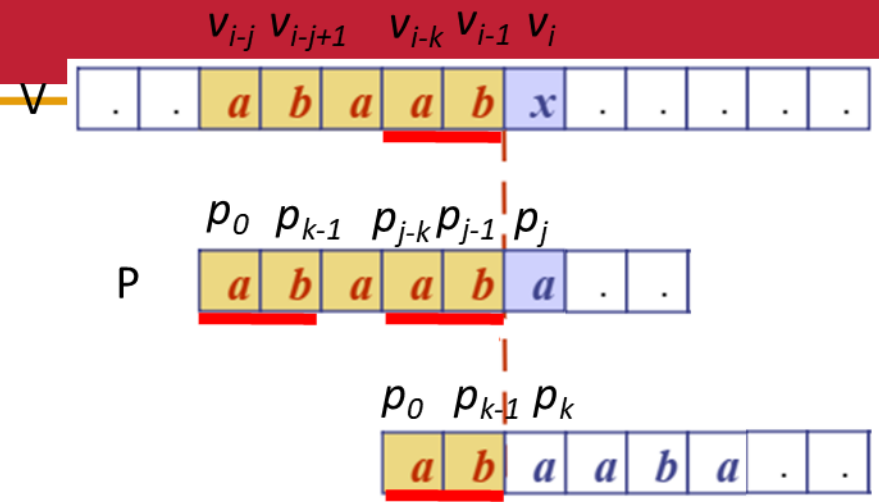
- Giải thuật Knuth-Morris-Pratt
 - k có thể được tính trước khi thực hiện việc tìm kiếm (thậm chí cả trước khi có nội dung văn bản)
 - Ví dụ
 - Cho mẫu **10110111**
 - $0 < k < j$, $[p_0 p_1 \dots p_{k-1}] = [p_{j-k} p_{j-k+1} \dots p_{j-1}]$

j	0	1	2	3	4	5	6	7
$p_0 \dots p_{j-1}$		1	10	101	1011----	10110---	101101--	1011011-
k			4 $k=1: p_0 \neq p_1$	1, 2 $k=1: p_0 = p_2$ $k=2: p_0 p_1 \neq p_1 p_2$	1, 2, 3	1, 2, 3, 4	1, 2, 3, 4, 5	1, 2, 3, 4, 5, 6
Chuỗi khớp	$\emptyset$	$\emptyset$	$\emptyset$	1 1xx----- xx1-----	1 1xxx----- xxx1-----	10 10xxx--- xxx10---	1 hoặc 101 101xxx-- xxx101--	1 hoặc 1011 1011xxx- xxx1011-

3. Tìm kiếm chuỗi con

- Nhận xét:

- Chọn k max
- Chọn $p_k \neq p_j$:



Mẫu 10110111

j	0	1	2	3	4	5	6	7
$p_0..p_{j-1}$		1	10	101	1011----	10110---	101101--	1011011-
k			1 $k=1: p_0 \neq p_1$	1, 2 $k=1: p_0=p_2$ $k=2: p_0p_1 \neq p_1p_2$	1, 2, 3	1, 2, 3, 4	1, 2, 3, 4, 5	1, 2, 3, 4, 5, 6
Chuỗi khớp	$\emptyset$	$\emptyset$	$\emptyset$	1 1xx----- xx1-----	1 1xxx----- xxx1-----	10 10xxx---	1 hoặc 101 101xxx-- xxx101--	1 hoặc 1011 1011xxx- xxx1011-
p_j	1	0	1	1	0	1	1	1
p_k				0	0	1	0, 1	0, 0
				$p_k \neq p_j$	Không thỏa mãn	Không thỏa mãn	$k=1 \Rightarrow p_k \neq p_j$	$p_k \neq p_j$
Chuỗi khớp mới	$\emptyset$	$\emptyset$	$\emptyset$	1	$\emptyset$	$\emptyset$	1	1011
k mới	0	0	0	1	0	0	1	4 (max)

3. Tìm kiếm chuỗi con

- Giải thuật Knuth-Morris-Pratt

- *Giải thuật tìm k:*

- Trả về mảng next[]: next[j]=k
- j: vị trí trong mẫu ($0 \leq j \leq M-1$)
- k: vị trí so khớp tiếp theo ($0 \leq k < j$) (chọn giá trị k lớn nhất có thể)

Bước 1: $k = j-1$;

Bước 2: Tìm k lớn nhất sao cho $p[k] \neq p[j]$;

Bước 3: So sánh hai chuỗi

$(p[0] \ p[1] \ .. \ p[k-1])$ và $(p[j-k] \ p[1] \ .. \ p[j-1])$

Nếu chúng bằng nhau thì

$next[j] = k$;

Kết thúc giải thuật;

Trái lại,

$k = k - 1$;

Lặp lại từ bước 2 của giải thuật;

3. Tìm kiếm chuỗi con

- Giải thuật Knuth-Morris-Pratt

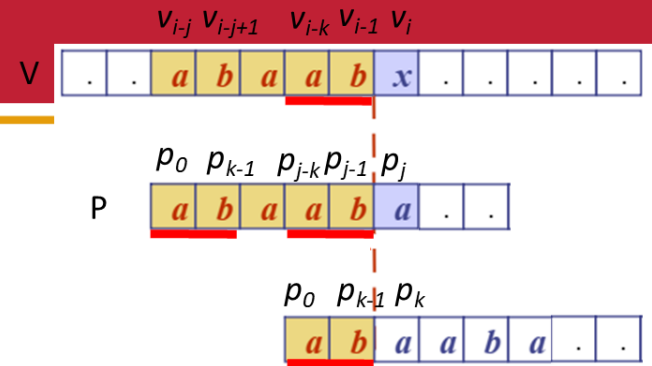
- Giải thuật tìm k:

- Cài đặt:

```
void initNext(char p[], int next[], int M ){
    int j,k;
    for (j=0; j<M; j++) next[j] = 0;
    for (j=2; j<M; j++){
        k = j-1;
        do {
            while (k>0 && p[k]==p[j]) k--;
            if (k>0){
                int i = 1;
                while (i<=k && p[k-i]==p[j-i]) i++;
                if (i>k){
                    next[j] = k;
                    break;
                }
            }
            k--;
        } while (k>0);
    }
}
```

B1: $k = j-1$;
B2: Tìm k lớn nhất sao cho $p[k] \neq p[j]$;
B3: So sánh hai chuỗi
 ($p[0]p[1]..p[k-1]$) và ($p[j-k]p[j-k+1]..p[j-1]$)
 Nếu chúng bằng nhau thì
 $next[j] = k$;
 Kết thúc giải thuật;
 Trái lại,
 $k = k - 1$;
 Lặp lại từ bước 2 của giải thuật;

3. Tìm kiếm chuỗi con



- Giải thuật Knuth-Morris-Pratt

- Cài đặt: `int KMPSearch(char V[], int N, char P[], int M ) {`

```
    if (N<M) return -1;
    int next[M];
    initNext(P, next, M);
    int i,j; i=j=0;
    do {
        while (j<M && V[i]==P[j]){i++; j++;}
        if (i<=N-M && j<M ) {
            if (j==0) i++;
            else j = next[j];
        }
    } while (i<=N-M && j<M) ;
    if (j==M) return (i-M);
    else return -1;
}
```

- Đánh giá giải thuật:

- Trường hợp xấu nhất, số thao tác so sánh kí tự của giải thuật chỉ là $M+N$
 $\Rightarrow O(N)$

4. Các giải thuật tìm kiếm trực tiếp

- Bài toán: Tìm phần tử bằng K trong dãy N số: $A = (a_0, a_1, \dots, a_{N-1})$
- Nguyên tắc lưu trữ, tìm kiếm:
 - Không thông qua so sánh
 - Thông qua địa chỉ ô nhớ tính trực tiếp trên các giá trị khóa (Tìm kiếm dựa vào/từ giá trị khóa)
- Hàm băm (hash function): công thức tính trực tiếp vị trí của phần tử cần tìm thông qua giá trị của nó
 - $y = h(k): k \rightarrow y$
 - $k \in D$: miền giá trị khóa của các phần tử
 - $y \in P$: miền giá trị của các địa chỉ của các phần tử
- Bài toán tìm kiếm $\Leftrightarrow$ Tính $h(K)$

3. Các giải thuật tìm kiếm trực tiếp

Sử dụng Hàm băm

- Lưu trữ:
 - Cho $k \rightarrow$ tính $h(k) \rightarrow$ lưu giá trị khóa k và bản ghi tương ứng vào ô nhớ địa chỉ $A_0 + h(k)$
 - Địa chỉ thực = $A_0 + h(k)$ (Quy ước: $A_0 = 0$)
- Tìm kiếm:
 - Cho $k \rightarrow$ tính $h(k) \rightarrow$ vào địa chỉ $A_0 + h(k)$, lấy ra giá trị khóa k và bản ghi tương ứng
 - Thao tác tìm kiếm: $O(1)$

Key table	Bảng địa chỉ	Bộ nhớ
	Chỉ số/STT	Key Info
2398	...	...
3487	398	2398
1728	...	...
4567	487	3487
...	...	...
...	567	4567
	...	...
	728	1728
	...	...

Ví dụ: $h(k) = k \bmod m$
 $m=1000$

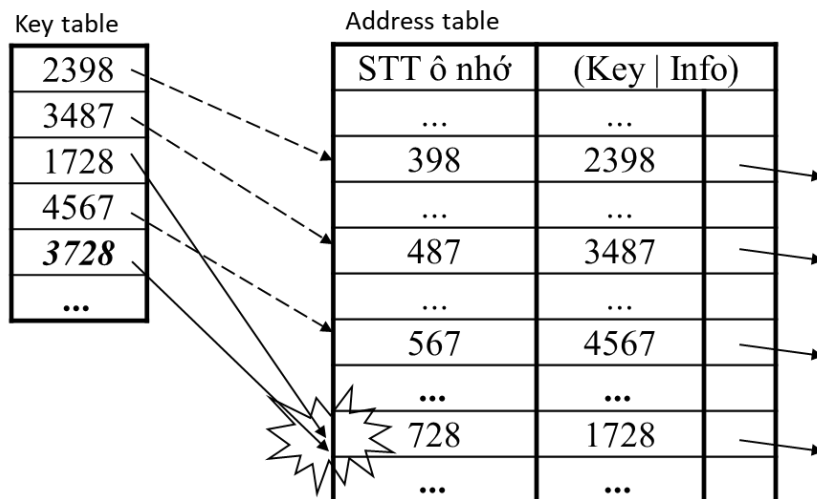
- Bảng khóa (key table): các giá trị khóa k
- Bảng địa chỉ - bảng băm (hash table) kích thước m

4. Các giải thuật tìm kiếm trực tiếp

- Hàm băm tốt:
 - đơn giản và có thể tính toán được trong thời gian ngắn
 - phân bố một tập các giá trị khóa một cách ngẫu nhiên và đều trên một tập các ô trong bảng
- Tính chất: hàm băm h thỏa mãn 2 điều kiện
 - **Đầy đủ:** miền xác định của h bao phủ hết miền giá trị các địa chỉ mà các phần tử có thể được lưu trữ (ánh xạ h là một toàn ánh)
 - Đảm bảo có thể tìm được bất kỳ phần tử nào dù cho nó nằm ở đâu trong miền lưu trữ
 - **Không đụng độ:**
 - Hiện tượng đụng độ: Xảy ra khi giá trị hàm băm trên hai khóa tìm kiếm khác nhau lại bằng nhau: $\exists K_1 \neq K_2: h(K_1) = h(K_2)$
 - Không đụng độ sẽ đảm bảo ánh xạ h là một đơn ánh

4. Các giải thuật tìm kiếm trực tiếp

- Xét cấu trúc lưu trữ danh sách:
 - Cấu trúc móc nối:
 - Động, địa chỉ các nút để lưu các phần tử là không cố định
 - Khó xác định được miền giá trị cho các địa chỉ của các nút
 - Cấu trúc mảng:
 - Địa chỉ của các phần tử có thể tính từ chỉ số của mảng
 - Tính không đụng độ thường không thể đạt được hoàn toàn (cần giảm khả năng đụng độ thấp nhất)



Kích thước của bảng địa chỉ có giới hạn => hiện tượng đụng độ địa chỉ

4. Các giải thuật tìm kiếm trực tiếp

- Các phương pháp xây dựng hàm băm
 - Xây dựng hàm địa chỉ cho các giá trị "*rải*" đều trên bảng
 - Phương pháp chia
 - Phương pháp nhân
 - Phương pháp phân đoạn
 - Đưa ra các biện pháp khắc phục đụng độ
 - Hash mở (open hashing): Biện pháp móc nối (Separate chaining)
 - Hash đóng (closed hashing): Biện pháp địa chỉ mở (open addressing)

- Phương pháp chia

- Đơn giản: $h(k) = k \bmod m$
- Số giá trị của $h(k)$ phụ thuộc vào m : kích thước bảng địa chỉ
- Hiện tượng đụng độ xảy ra khi hai giá trị khóa đồng dư với m
- Yêu cầu: Chọn m sao cho $h(k)$ rải đều
 - Nếu m nguyên tố $\Rightarrow$ tốt
- Cải tiến:
 - $h(k) = k \bmod m^*$, với m^* là số nguyên tố lớn nhất $\leq m$

Các phương pháp xây dựng hàm băm

- Phương pháp nhân

- $h(k) = \lfloor m * ((k * A) \bmod 1) \rfloor$
 - A: giá trị nằm trong khoảng (0, 1)
 - m: kích thước bảng địa chỉ
 - A thường được chọn phụ thuộc vào tính chất dữ liệu được băm.
($A_{\text{Knuth}} = (\sqrt{5} - 1)/2$)

- Phương pháp trung bình bình phương (mid-square)

- Xác định $h(k)$ thông qua các chữ số liên tục ở giữa số k^2
 - Tính k^2 , chọn r chữ số ở giữa của kết quả k^2 .
 - Ví dụ: $m=1000$

-> cần chọn $r=3$

k	k*k	h(k)
2398	5750404	504
3487	12159169	591
1728	2985984	859
4567	20857489	574
3728	13897984	979

Các phương pháp xây dựng hàm băm

- Phương pháp phân đoạn

- Áp dụng khi khóa có kích thước lớn
- Chia khóa thành các đoạn có độ dài như nhau = độ dài địa chỉ
- Phối hợp các đoạn
 - Ví dụ: cộng lại, chọn một vài vị trí và ghép lại

- Các kỹ thuật phân đoạn

- Tách (splitting)
 - Tách khóa k từ phải qua trái thành các đoạn có độ dài = $m \Rightarrow$ các số $k_1, k_2, \dots, k_n$
 - $h(k) = k_1 + k_2 + \dots + k_n$
- Gấp (folding)
 - Gấp theo các đường biên (giống gấp giấy)
 - Tính tổng các đoạn

Các phương pháp xây dựng hàm băm

- Ví dụ Phương pháp phân đoạn

- $k = 34289421$

- Kích thước bảng địa chỉ 1000

- => Độ dài địa chỉ = 3

- Tách: => $k_1=421, k_2=289, k_3=034$

Tách 421

289

034

$h(k)=$ 744

- Gấp: gấp theo biên 034|289|421

- => $k_1=124, k_2=289, k_3=430$

Gấp 124

289

430

$h(k)=$ 843

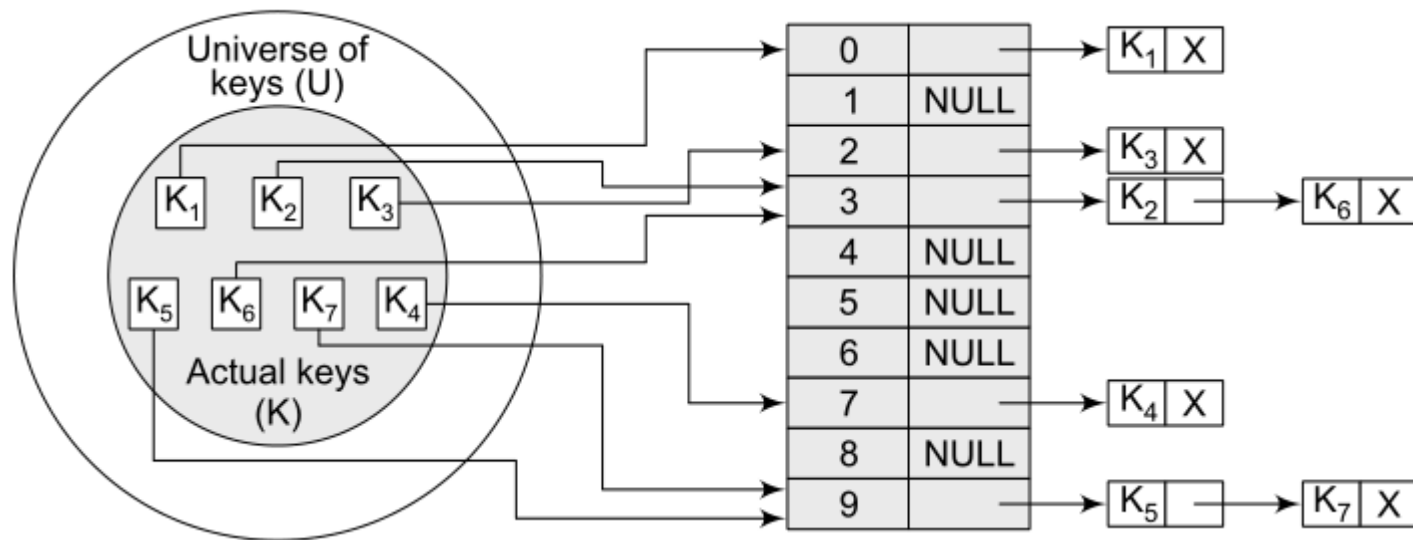
- Vấn đề đụng độ:
 - Tập giá trị khóa thường rất lớn trong khi bảng địa chỉ thường có kích thước hạn chế => các khóa có giá trị khác nhau lại có giá trị địa chỉ giống nhau => đụng độ
 - Đụng độ là khi ô nhớ có địa chỉ tính ra đã bị chiếm => cần tìm một ô nhớ khác để lưu các giá trị khóa đó
 - Vấn đề đụng độ có thể xảy ra liên tục, dây chuyền nếu không có chiến lược khắc phục
- Các chiến lược khắc phục đụng độ (rehashing)
 - Biện pháp móc nối (Separate chaining)
 - Biện pháp địa chỉ mở (open addressing)
 - Phương pháp thử tuyến tính (băm lại tuyến tính)
 - Phương pháp thử bình phương (băm lại bình phương)
 - Quy ước: bảng khóa gồm n khóa khác nhau, bảng địa chỉ kích thước m

Các biện pháp khắc phục độ

- Biện pháp móc nối (Separate chaining)

- Nguyên tắc lưu trữ & tìm kiếm

- Dùng CTLT móc nối để lưu trữ và tìm kiếm
- Mỗi bản ghi trong bảng băm chứa con trỏ tới danh sách liên kết chứa tất cả các giá trị khóa được lưu trữ tại địa chỉ này.



Các biện pháp khắc phục độ

• Biện pháp móc nối

• Nguyên tắc lưu trữ & tìm kiếm

- Dùng CTLT móc nối để lưu trữ và tìm kiếm
- Mỗi bản ghi trong bảng băm chứa con trỏ trỏ tới danh sách liên kết chứa tất cả các giá trị khóa được lưu trữ tại địa chỉ này.
- Tìm kiếm 1 khóa: duyệt qua danh sách liên kết được đặt tại địa chỉ $h(k)$
- Thêm khóa: bổ sung khóa vào cuối dslk
- Xóa khóa: tìm và xóa key khỏi dslk

• Minh họa: Thêm các khóa vào bảng hash móc nối có kích thước = 7.

- $h(k) = k \bmod m, m=7$

k	$h(k)$
123	4
321	6
234	3
432	5
345	2
543	4
678	6

	Giá trị	P
0		
1		
2		→ 345
3		→ 234
4		→ 123 → 543
5		→ 432
6		→ 321 → 678

Kỹ thuật móc nối ngoài

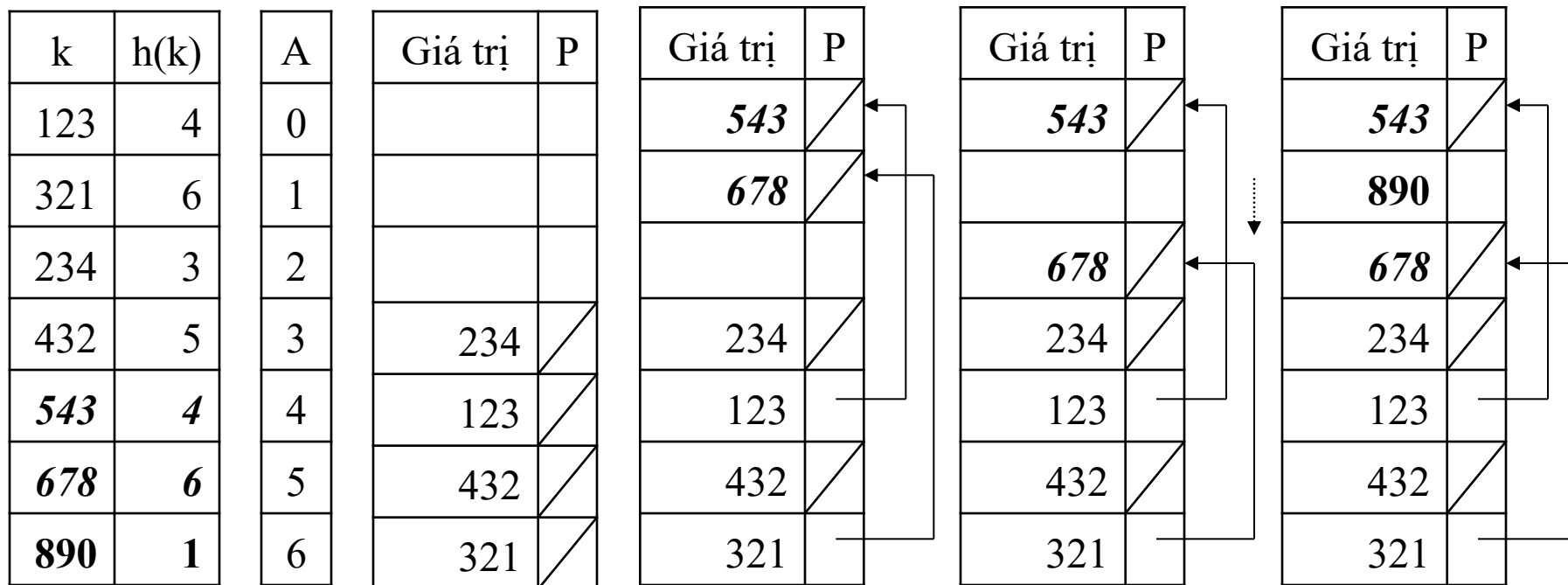
Nhận xét: Tốn bộ nhớ

Các biện pháp khắc phục đụng độ

• Biện pháp móc nối

• Kỹ thuật móc nối trong:

- Dùng miền nhớ ở trong bảng địa chỉ để lưu các giá trị, sử dụng móc nối để liên kết các “đụng độ”
- Chỉ dùng thêm miền nhớ mới khi xảy ra hiện tượng tràn (không còn ô nhớ trống trong bảng hash)



- Biện pháp địa chỉ mở (open addressing)
 - Nguyên tắc lưu trữ & tìm kiếm nếu có đụng độ
 - Thử các địa chỉ theo một cách nhất định để tìm vị trí mới (dò theo trình tự)
 - Nếu đi hết bảng địa chỉ mà không tìm ra thì quay lại từ đầu bảng
 - Nếu không tìm thấy chỗ trống => overflow (tràn bảng)
 - Các phương pháp dò:
 - Dò tuyến tính
 - Dò bậc hai
 - Băm kép (double hashing)
 - Băm lại (rehashing)

- Biện pháp địa chỉ mở - *Phương pháp dò tuyến tính (Linear Probing)*
 - Giả sử cần lưu 1 bản ghi mới có khóa = k .
 - Ô nhớ tại $h(k)$ đã chứa một bản ghi khác
→ Tìm ô nhớ khác tại địa chỉ $h(k,i)$ để lưu bản ghi mới này
 - Tổng quát: $h(k,i) = [h'(k)+i] \bmod m$
 - m : kích thước bảng địa chỉ
 - $h'(k) = (k \bmod m)$
 - i : $0..m-1$
 - $i=0$: Tìm vị trí đầu tiên $h(k)$. Nếu vị trí này đã chứa bản ghi => tăng i lên 1
 - Chưa tìm được vị trí trống, tiếp tục tăng i lên 1.
 - Nếu cuối cùng vẫn không tìm được vị trí mới cho bản ghi → overflow

Các biện pháp khắc phục đụng độ

- Biện pháp địa chỉ mở - *Phương pháp dò tuyến tính (Linear Probing)*

- Ví dụ: Cho bảng hash có kích thước = 10. Thêm các khóa 72, 27, 36, 24, 63, 81, 92 ... vào bảng

- $h(k) = k \bmod m, m = 10$

- $h(k,i) = [k \bmod m + i] \bmod m \ (i = 1..m)$

- **Ví dụ: Tìm kiếm !**

=-1: sentinel value: *free box*

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Thêm k = 72 $\rightarrow h(k)=2$

Thêm k = 27 $\rightarrow h(k)=7$

Thêm k = 36 $\rightarrow h(k)=6$

...

...

Thêm k = 24 $\rightarrow h(k)=4$

Thêm k = 63 $\rightarrow h(k)=3$

Thêm k = 81 $\rightarrow h(k)=1$

0	1	2	3	4	5	6	7	8	9
0	81	72	63	24	-1	36	27	-1	-1

Thêm k = 92 $\rightarrow h(k)=2 \rightarrow$ đụng độ

Dò: $h(92,1) = (92 \bmod 10 + 1) \bmod 10 = 3$: occupied

$h(92,2) = (92 \bmod 10 + 2) \bmod 10 = 4$: occupied

$h(92,3) = (92 \bmod 10 + 3) \bmod 10 = 5$: free

$\rightarrow$ thêm vào vị trí chỉ số 5

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	92	36	27	-1	-1

- Biện pháp địa chỉ mở - Phương pháp dò tuyến tính (*Linear Probing*)
 - Nhận xét đánh giá
 - Hiện tượng hội tụ (clustering):
 - Việc dò tuyến tính thực hiện từ đầu – cuối: các vị trí rỗng là các vị trí đứng sau các các vị trí không rỗng => Thêm khóa vào vị trí này tạo thành các cụm phần tử liên tiếp không rỗng.
 - Việc dò vị trí rỗng cho phần tử khóa đụng độ tiếp theo sẽ phải dò qua các cụm hội tụ này
 - Càng nhiều đụng độ xảy ra, càng phải dò qua nhiều cụm kích thước lớn => tốc độ càng chậm

Các biện pháp khắc phục đụng độ

- Biện pháp địa chỉ mở
 - Hướng giải quyết
 - Công thức mới:
 - Dò bậc hai: $h(k, i) = [h'(k) + c_1i + c_2i^2] \bmod m$
 $i: 0..m-1, c_1, c_2: \text{const}, \neq 0$
 - Băm kép (double hashing): $h(k, i) = [h_1(k) + ih_2(k)] \bmod m$
 $h_1(k) = k \bmod m, h_2(k) = k \bmod m', m' = m-1 \text{ hoặc } m-2$
 - Băm lại (rehashing): Tạo bảng băm mới có kích thước lớn hơn bảng ban đầu, bố trí lại các key vào bảng mới

- Đánh giá

- Đánh giá hệ số tải trọng

- m = kích thước bảng băm
 - n = số khóa được chèn vào bảng băm ($n \leq m$)
 - Hệ số tải trọng $\alpha = n / (m+1) (<1)$

- Đánh giá số bước thực hiện

- Giả sử các khoá xuất hiện đồng đều trong bảng băm
 - Ước lượng E : số lần thử trung bình để truy cập một khóa ngẫu nhiên trong bảng

⇔ Tìm thấy vị trí trống mới cho nó hoặc tìm thấy khóa đó đã lưu trong bảng (dựa trên lý thuyết xác suất):

- Biện pháp móc nối

$$E = 1 + \alpha/2$$

a) Kỹ thuật móc nối ngoài (External Link):

- Biện pháp địa chỉ mở $E = -\frac{1}{\alpha} \ln(1 - \alpha)$

b) Phép thử ngẫu nhiên:

c) Phép thử tuyến tính: $E = \frac{1 - \alpha/2}{1 - \alpha}$

Các biện pháp khắc phục độ

- Nhận xét

Biện pháp móc nối	Biện pháp địa chỉ mở
Thực thi đơn giản	Yêu cầu nhiều tính toán
Bảng băm không bao giờ đầy, luôn có thể thêm khóa vào bảng	Bảng có thể đầy (overflow)
Ít nhạy cảm hơn với hàm băm hoặc các hệ số tải	Cần xử lý thêm vấn đề hội tụ, hệ số tải
Chủ yếu được sử dụng khi không biết số lượng và tần suất các khóa có thể được chèn hoặc xóa	Sử dụng khi biết số lượng và tần suất các khóa có thể được chèn hoặc xóa
Hiệu quả sử dụng tính chất bộ đệm kém	Cung cấp hiệu quả bộ đệm cao do tất cả được lưu trong cùng 1 bảng
Lãng phí không gian (Một số phần của bảng băm không bao giờ được sử dụng)	Tận dụng khe nhớ khi chưa được sử dụng
Sử dụng thêm không gian cho các liên kết.	Không sử dụng liên kết

- 1. Viết chương trình phần mềm tìm kiếm 1 phần tử trong một dãy số nguyên bất kỳ, dùng phương pháp tìm kiếm nhị phân
- 2. Viết các hàm khởi tạo, thêm, xóa, tìm kiếm 1 giá trị khóa trong một bảng băm áp dụng phương pháp khắc phục đụng độ dạng móc nối
- 3. Thực hiện tìm kiếm chuỗi con P trong chuỗi văn bản V bằng giải thuật Knuth-Morris-Pratt, với $V = \text{abdcababdcabdc}$ và $P = \text{bdcabdc}$.